

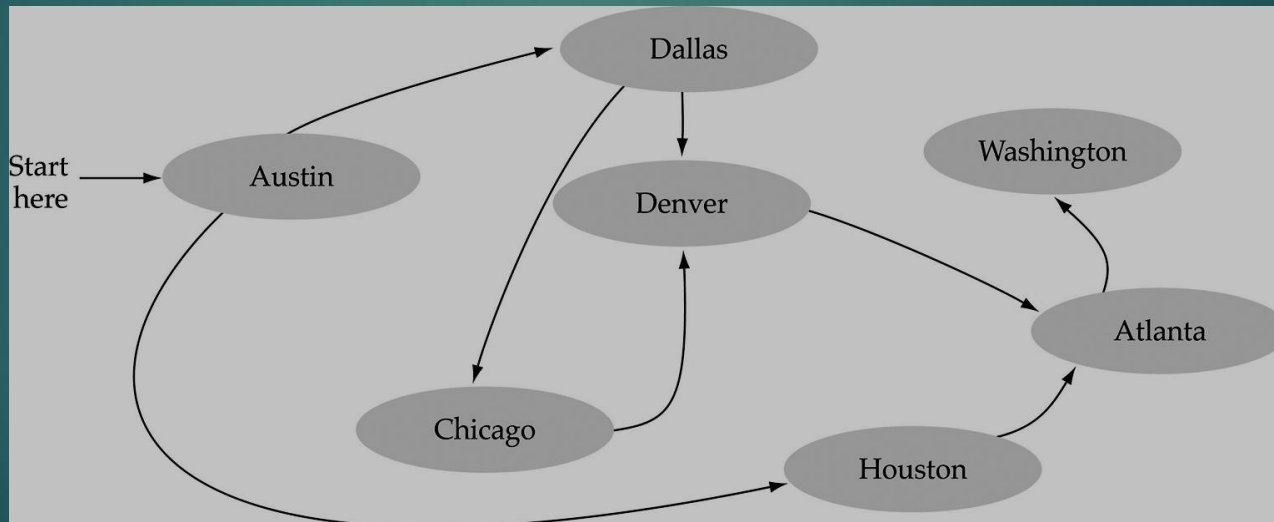


Week 15

Graph Data Structure

What is a graph?

- A data structure that consists of a set of nodes (*vertices*) and a set of edges that relate the nodes to each other
- The set of edges describes relationships among the vertices



Formal definition of graphs

- A graph G is defined as follows:

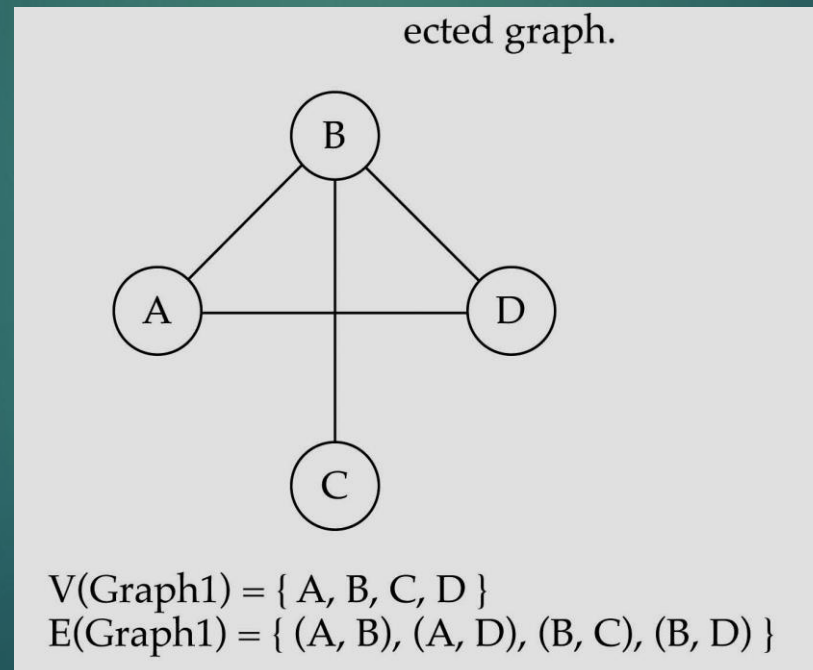
$$G=(V,E)$$

$V(G)$: a finite, nonempty set of vertices

$E(G)$: a set of edges (pairs of vertices)

Directed vs. undirected graphs

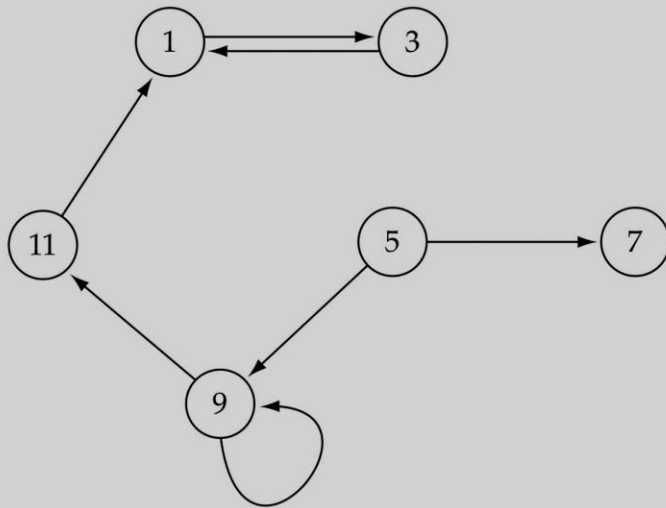
- When the edges in a graph have no direction, the graph is called *undirected*



Directed vs. undirected graphs (cont.)

- When the edges in a graph have a direction, the graph is called *directed* (or *digraph*)

(b) Graph2 is a directed graph.



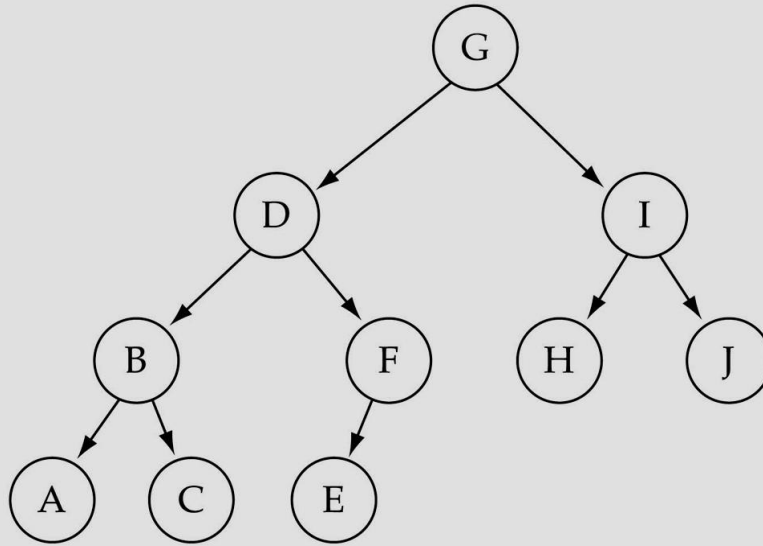
$V(\text{Graph2}) = \{ 1, 3, 5, 7, 9, 11 \}$

$E(\text{Graph2}) = \{(1,3), (3,1), (5,9), (9,11), (5,7), (9,9), (11,1)\}$

Trees vs graphs

- Trees are special cases of graphs!!

(c) Graph3 is a directed graph.

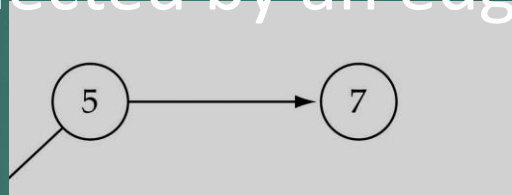


$V(\text{Graph3}) = \{ A, B, C, D, E, F, G, H, I, J \}$

$E(\text{Graph3}) = \{ (G, D), (G, I), (D, B), (D, F), (I, H), (I, J), (B, A), (B, C), (F, E) \}$

Graph terminology

- Adjacent nodes: two nodes are adjacent if they are connected by an edge



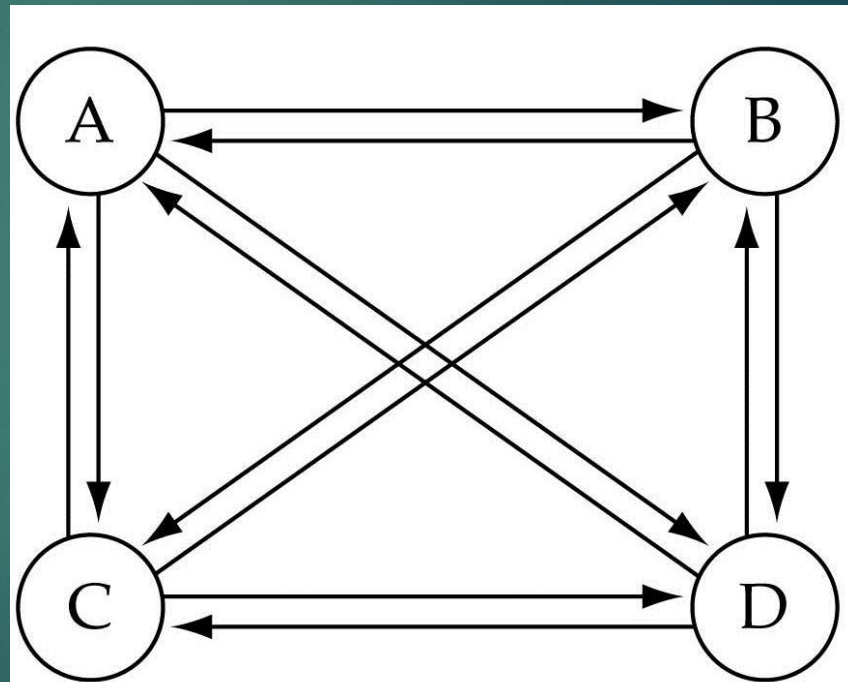
- Path: a sequence of vertices that connect two nodes in a graph
- Complete graph: a graph in which every vertex is directly connected to every other vertex

Graph terminology (cont.)

- What is the number of edges in a complete directed graph with N vertices?

$$N * (N-1)$$

$$O(N^2)$$



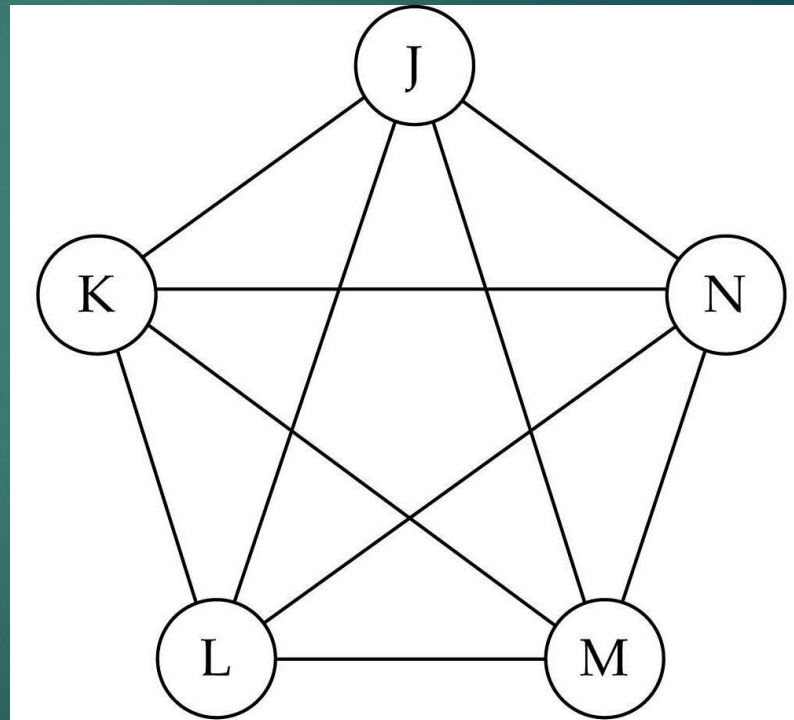
(a) Complete directed graph.

Graph terminology (cont.)

- What is the number of edges in a complete undirected graph with N vertices?

$$N * (N-1) / 2$$

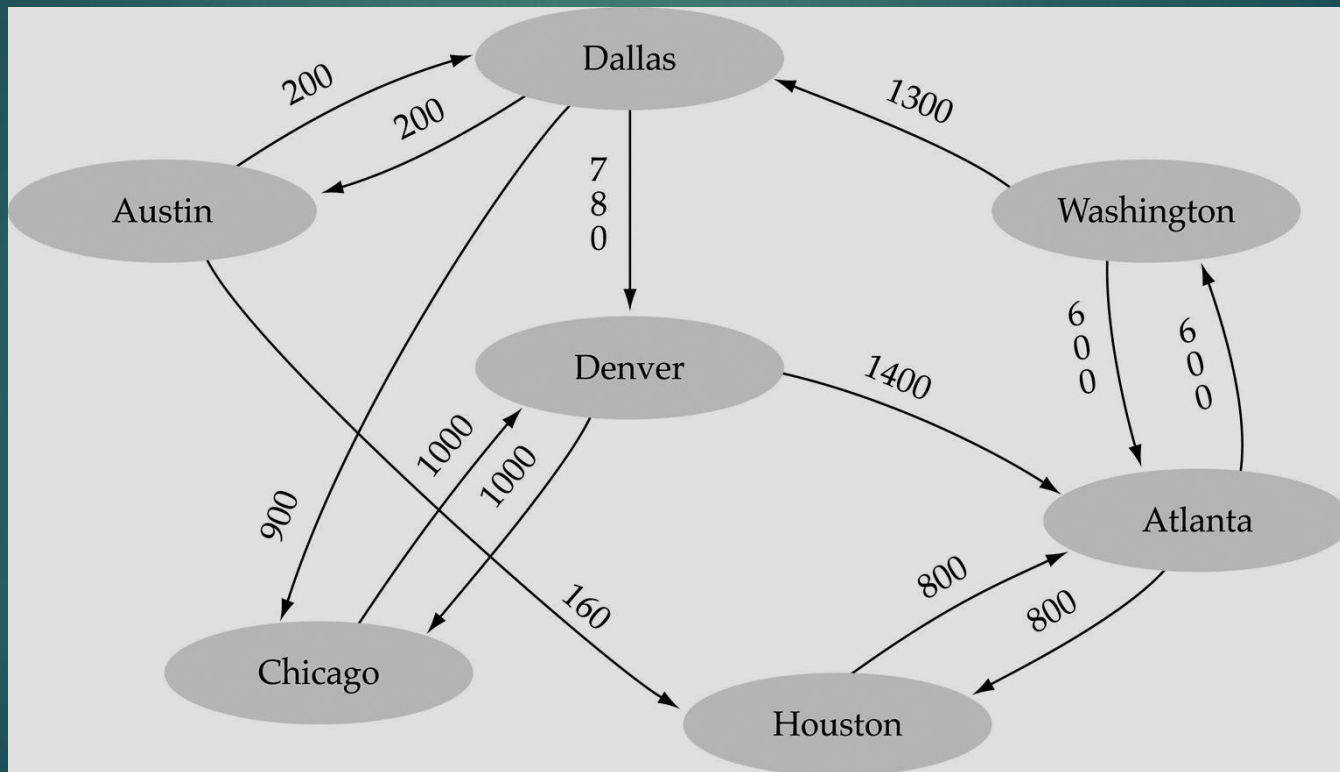
$$O(N^2)$$



(b) Complete undirected graph.

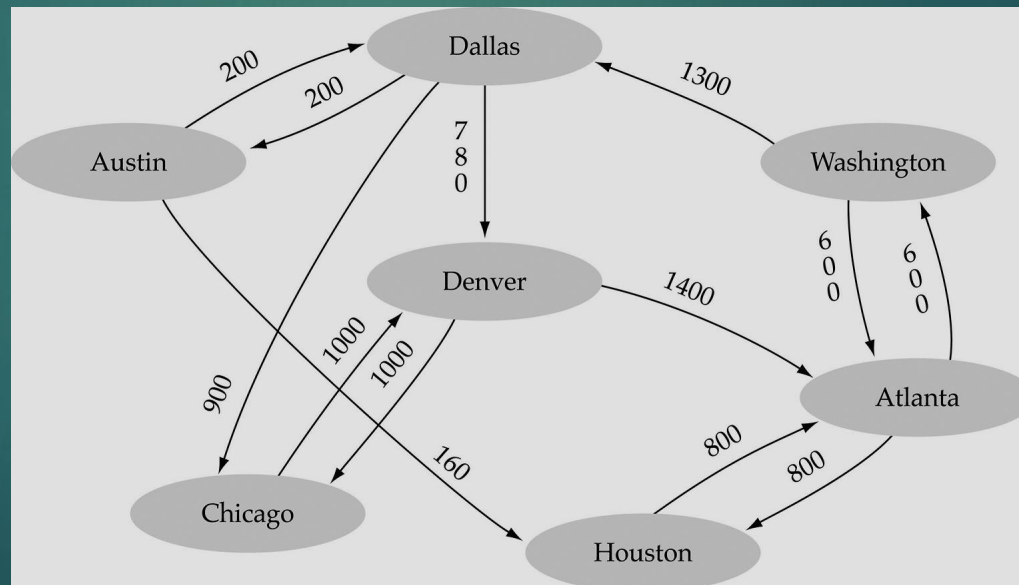
Graph terminology (cont.)

- Weighted graph: a graph in which each edge carries a value



Graph Representation

- Array-based implementation
 - A 1D array is used to represent the vertices
 - A 2D array (adjacency matrix) is used to represent the edges



graph

.numVertices 7

.vertices

[0]	"Atlanta"	"
[1]	"Austin"	"
[2]	"Chicago"	"
[3]	"Dallas"	"
[4]	"Denver"	"
[5]	"Houston"	"
[6]	"Washington"	"
[7]		
[8]		
[9]		

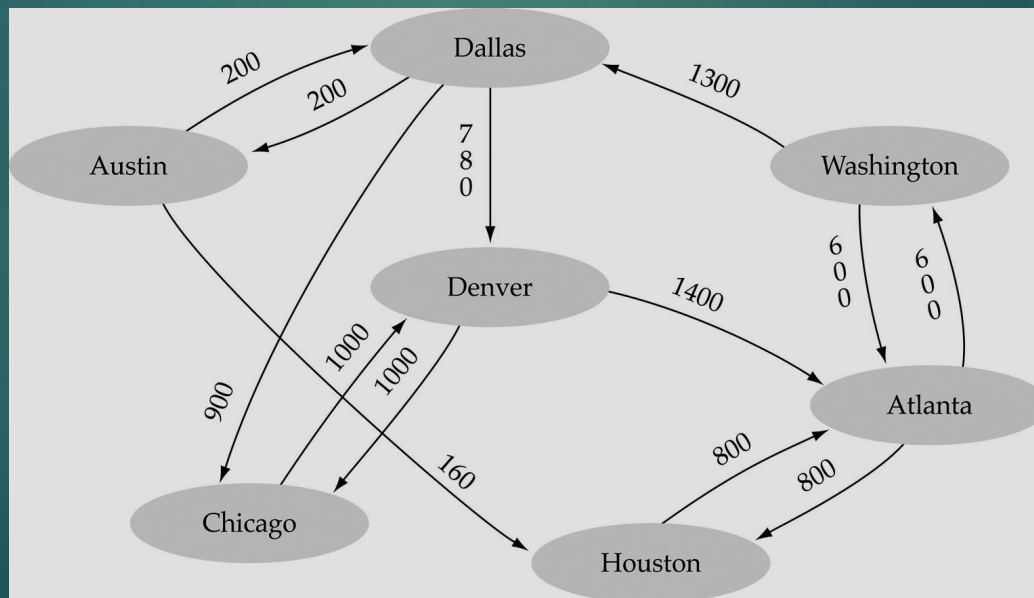
.edges

[0]	0	0	0	0	0	800	600	•	•	•
[1]	0	0	0	200	0	160	0	•	•	•
[2]	0	0	0	0	1000	0	0	•	•	•
[3]	0	200	900	0	780	0	0	•	•	•
[4]	1400	0	1000	0	0	0	0	•	•	•
[5]	800	0	0	0	0	0	0	•	•	•
[6]	600	0	0	1300	0	0	0	•	•	•
[7]	•	•	•	•	•	•	•	•	•	•
[8]	•	•	•	•	•	•	•	•	•	•
[9]	•	•	•	•	•	•	•	•	•	•
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

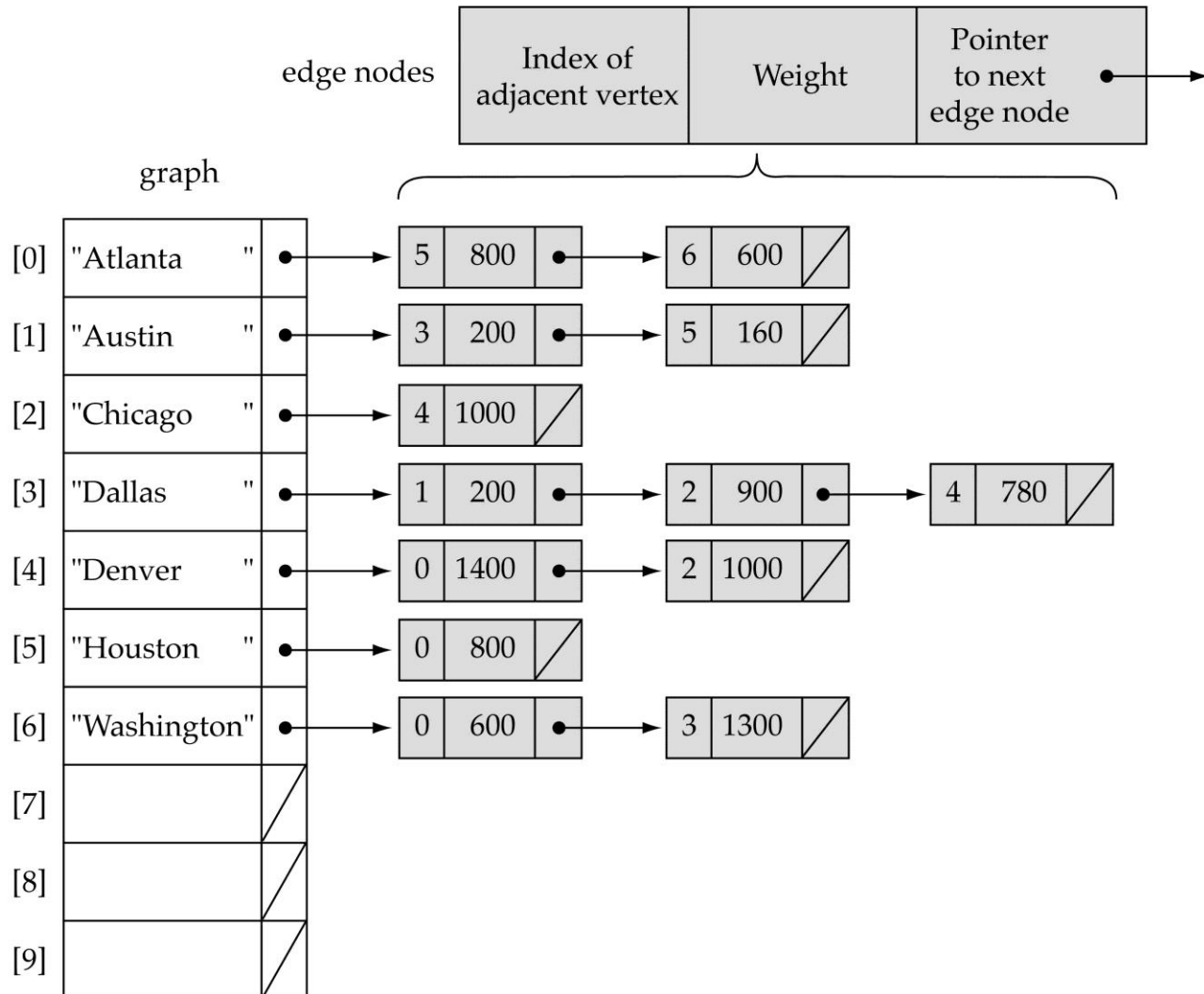
(Array positions marked '•' are undefined)

Graph Representation (cont.)

- Linked-list implementation
 - A 1D array is used to represent the vertices
 - A list is used for each vertex v which contains the vertices which are adjacent from v (adjacency list)



(a)



Adjacency matrix vs. adjacency list representation

- **Adjacency matrix**
 - Good for dense graphs
 - Connectivity between two vertices can be tested quickly
- **Adjacency list**
 - Good for sparse graphs
 - Vertices adjacent to another vertex can be found quickly



Graph searching

- Problem: find a path between two nodes of the graph (e.g., Austin and Washington)
- Methods: Depth-First-Search (DFS) or Breadth-First-Search (BFS)

BFS

- ▶ **Initialization:** Enqueue the given source vertex into a queue and mark it as visited.
- 1. **Exploration:** While the queue is not empty:
 - 1. Dequeue a node from the queue and visit it (e.g., print its value).
 - 2. For each unvisited neighbor of the dequeued node:
 - 1. Enqueue the neighbor into the queue.
 - 2. Mark the neighbor as visited.
- 2. **Termination:** Repeat step 2 until the queue is empty.

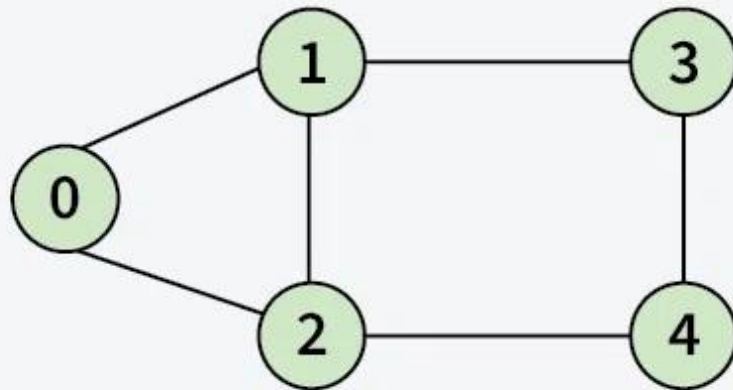
BFS Using Queue

1. Create a queue Q.
2. Mark all nodes as unvisited.
3. Enqueue starting node s , mark visited.
4. While Q is not empty:
 - $u = Q.dequeue()$
 - For every adjacent v of u :
 - If v not visited:
 - mark visited
 - enqueue v

BFS

01
Step

Initially queue and visited array are empty.



Visited:

--	--	--	--	--

Queue:

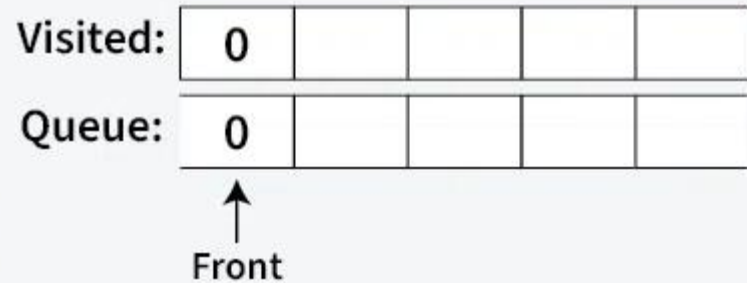
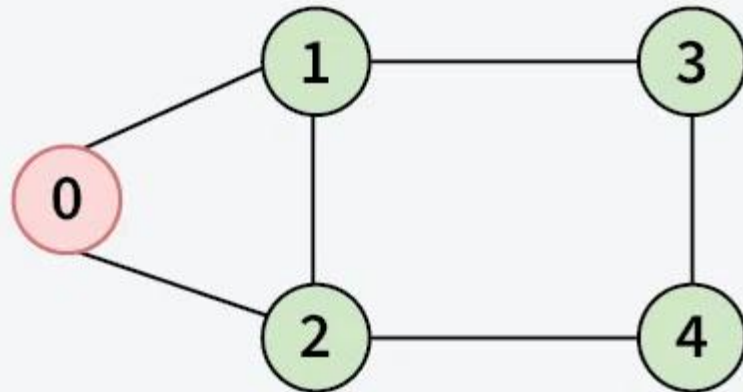
--	--	--	--	--

↑
Front

BFS on Graph

02
Step

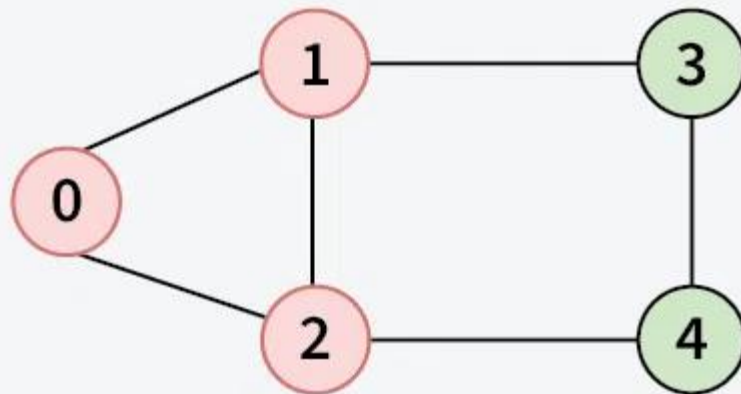
Push 0 into queue and mark it visited.



BFS on Graph

03
Step

Remove 0 from the front of queue and visit the unvisited neighbours and push them into queue.

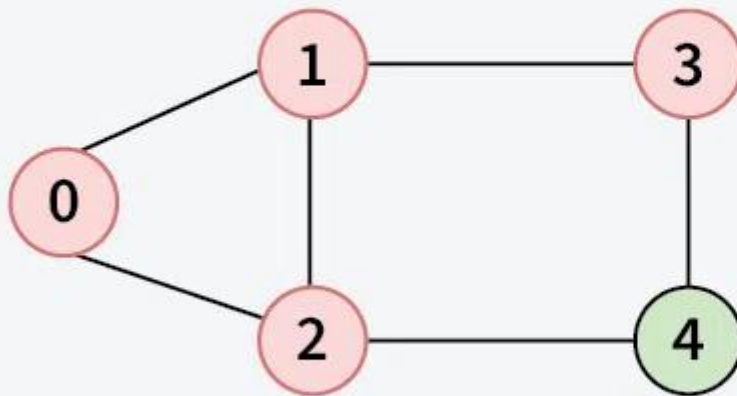


Visited:	0	1	2		
Queue:	0	1	2		
		↑			
		Front			

BFS on Graph

04
Step

Remove node 1 from the front of queue and visit the unvisited neighbours and push them into queue.



Visited:

0	1	2	3	
---	---	---	---	--

Queue:

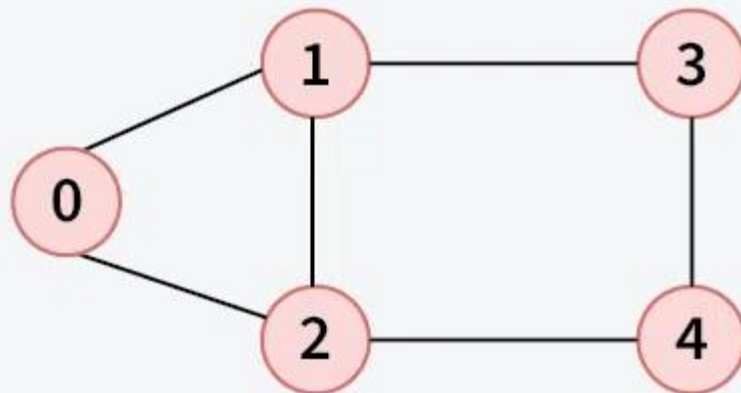
1	2	3		
---	---	---	--	--

↑
Front

BFS on Graph

05
Step

Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.



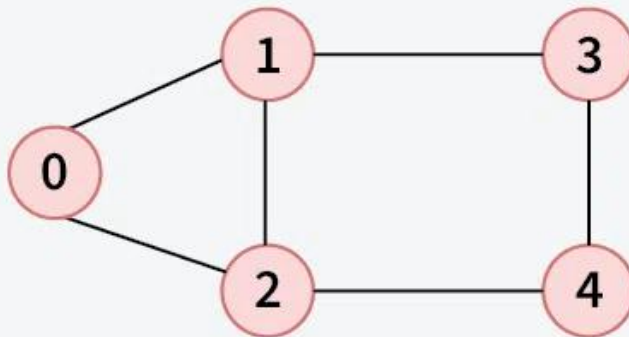
Visited:	0	1	2	3	4
Queue:	2	3	4		

↑
Front

BFS on Graph

06
Step

Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue.



Visited:

0	1	2	3	4
---	---	---	---	---

Queue:

3	4			
---	---	--	--	--

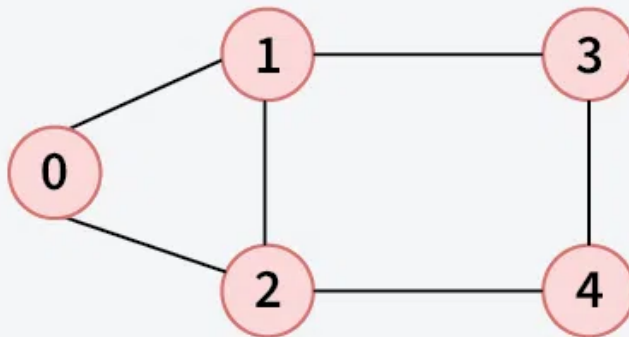
↑
Front

All neighbors of node 3 have been visited, proceed to the next node in the queue.

BFS on Graph

07
Step

Remove node 4 from the front of queue and visit the unvisited neighbours and push them into queue.



Visited:

0	1	2	3	4
---	---	---	---	---

Queue:

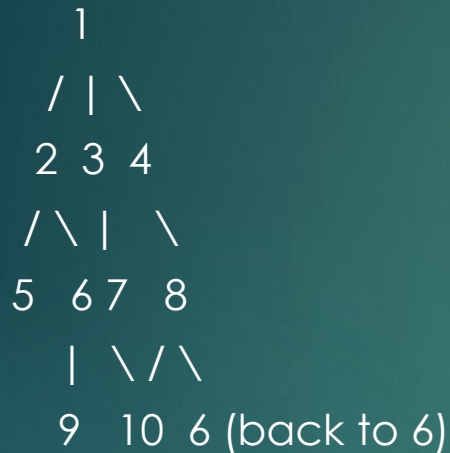
4				
---	--	--	--	--

↑
Front

All neighbors of node 4 have been visited, proceed to the next node in the queue.

BFS- Another example UDG

- ▶ Graph with **10 nodes**, undirected:



1: 2, 3, 4
2: 1, 5, 6
3: 1, 7
4: 1, 8
5: 2
6: 2, 9
7: 3, 10
8: 4, 10, 6
9: 6
10: 7, 8

- ▶ **Adjacency List**
- ▶ (We will use this order when visiting neighbors.)

Algorithm Reminder

- ▶ Use **Queue**
- ▶ Visit node when dequeued
- ▶ Push all **unvisited neighbours**

Initialization

- ▶ Queue: [1]
- ▶ Visited: {1}
- ▶ Output:

STEP 1

- ▶ Dequeue → 1
- ▶ Neighbors: 2, 3, 4 → Enqueue
- ▶ Queue: [2, 3, 4]
- ▶ Visited: {1,2,3,4}
- ▶ Output: 1

STEP 2

- ▶ Dequeue → 2
- ▶ Neighbors: 1, 5, 6 → Add unvisited → 5,6
- ▶ Queue: [3, 4, 5, 6]
- ▶ Visited: {1,2,3,4,5,6}
- ▶ Output: 1 2

STEP 3

- ▶ Dequeue → 3
- ▶ Neighbors: 1,7 → Add 7
- ▶ Queue: [4, 5, 6, 7]
- ▶ Visited: {1,2,3,4,5,6,7}
- ▶ Output: 1 2 3

STEP 4

- ▶ Dequeue → 4
- ▶ Neighbors: 1,8 → Add 8
- ▶ Queue: [5, 6, 7, 8]
- ▶ Visited: {1,2,3,4,5,6,7,8}
- ▶ Output: 1 2 3 4

STEP 5

- ▶ Dequeue → 5
- ▶ Neighbors: 2 → visited
- ▶ Queue: [6, 7, 8]
- ▶ Output: 1 2 3 4 5

STEP 6

- ▶ Dequeue → 6
- ▶ Neighbors: 2,9 → Add 9
- ▶ Queue: [7, 8, 9]
- ▶ Visited: {1,2,3,4,5,6,7,8,9}
- ▶ Output: 1 2 3 4 5 6

STEP 9

- ▶ Dequeue → **9**
- ▶ Only neighbor: 6 (visited)
- ▶ Queue: [10]
- ▶ Output: 1 2 3 4 5 6 7 8 9

STEP 10

- ▶ Dequeue → **10**
- ▶ Neighbors: 7, 8 → visited
- ▶ Queue: []
- ▶ Output: 1 2 3 4 5 6 7 8 9 10

FINAL BFS TRAVERSAL

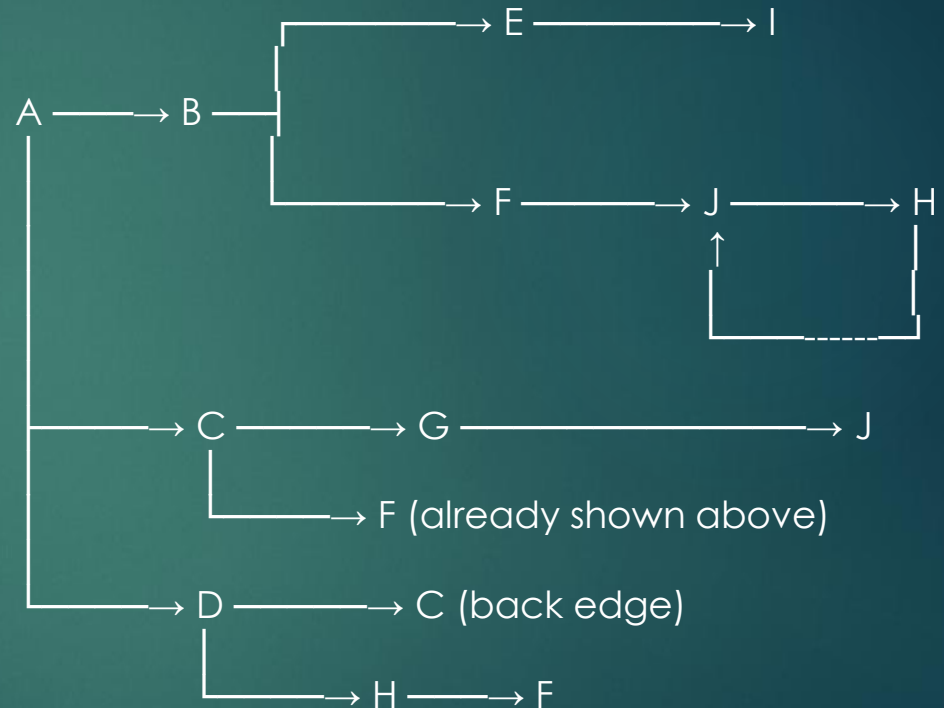
- ▶ 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

BFS- Another example DG

We use nodes: **A, B, C, D, E, F, G, H, I, J**

Adjacency List (Directed Edges)

- ▶ $A \rightarrow B, C, D$
- ▶ $B \rightarrow E, F$
- ▶ $C \rightarrow F, G$
- ▶ $D \rightarrow C, H$
- ▶ $E \rightarrow I$
- ▶ $F \rightarrow I, J$
- ▶ $G \rightarrow J$
- ▶ $H \rightarrow F$
- ▶ $I \rightarrow$ (no outgoing edges)
- ▶ $J \rightarrow H$



Algorithm Reminder

- ▶ Use **Queue**
- ▶ Visit node when dequeued
- ▶ Push all **unvisited neighbours**

BFS- Another example

Step	Queue	Visited	Dequeued	Action
1	[A]	{}	—	start
2	[]	{A}	A	Add B, C, D
3	[B, C, D]	{A}	—	—
4	[C, D]	{A, B}	B	Add E, F
5	[D, E, F]	{A, B, C}	C	Add F*, G
6	[E, F, G]	{A, B, C, D}	D	Add C*, H
7	[F, G, H]	{A, B, C, D, E}	E	Add I
8	[G, H, I]	{A, B, C, D, E, F}	F	Add I*, J
9	[H, I, J]	{A, B, C, D, E, F, G}	G	Add J*
10	[I, J]	{A, B, C, D, E, F, G, H}	H	Add F*
11	[J]	{A, B, C, D, E, F, G, H, I}	I	no edges
12	[]	{all}	J	Add H*

FINAL BFS ORDER

A → B → C → D → E → F → G → H → I → J

BFS

- ▶ **Time Complexity:** $O(V+E)$, where V is the number of nodes and E is the number of edges.
Auxiliary Space: $O(V)$

Depth First Search

- ▶ Depth-first search (DFS) is an algorithm for searching the vertices of a graph, that works by exploring as far into the graph as possible before backtracking
- ▶ It uses **Stack** (explicit or recursion).
- ▶ Good for **detecting cycles**
- ▶ Good for **connected components**
- ▶ Good for **topological sorting**

Depth First Search Using Stack

DFS(Graph, start):

- create stack S

- push(start)

- while S not empty:

 - u = pop(S)

 - if not visited[u]:

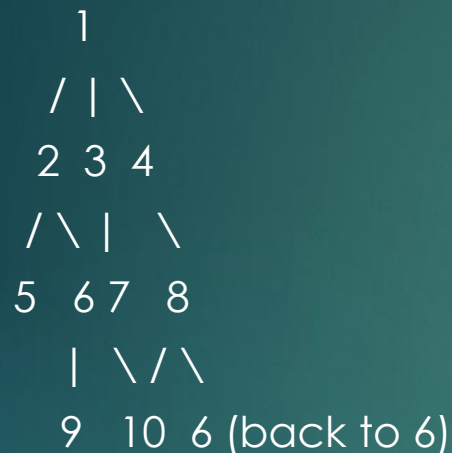
 - visited[u] = true

 - print u

 - push all neighbors of u (in reverse order)

DFS- undirected Graph Example

- ▶ Graph with **10 nodes**, undirected:



1: 2, 3, 4
2: 1, 5, 6
3: 1, 7
4: 1, 8
5: 2
6: 2, 9
7: 3, 10
8: 4, 10, 6
9: 6
10: 7, 8

- ▶ **Adjacency List**
- ▶ (We will use this order when visiting neighbors.)

Algorithm Reminder

- ▶ Use **Stack**
- ▶ Process node when **popped**
- ▶ Push neighbors in **reverse order** to maintain correct order

Initialization

- ▶ Stack: [1]
- ▶ Visited: { }
- ▶ Output:

STEP 1

- ▶ Pop → **1**
- ▶ Visited: {1}
- ▶ Output: 1
- ▶ Neighbors of 1: 2,3,4 → push reverse: 4,3,2
- ▶ Stack: [4, 3, 2]

STEP 2

- ▶ Pop → **2**
- ▶ Visited: {1,2}
- ▶ Output: 1 2
- ▶ Neighbors of 2: 1,5,6 → push reverse: 6,5
- ▶ Stack: [4, 3, 6, 5]

STEP 3

- ▶ Pop → **5**
- ▶ Visited: {1,2,5}
- ▶ Output: 1 2 5
- ▶ Neighbor: 2 (visited) → no push
- ▶ Stack: [4, 3, 6]

STEP 4

- ▶ Pop → **6**
- ▶ Visited: {1,2,5,6}
- ▶ Output: 1 2 5 6
- ▶ Neighbors: 2,9 → push 9
- ▶ Stack: [4, 3, 9]

STEP 5

- ▶ Pop → **9**
- ▶ Visited: {1,2,5,6,9}
- ▶ Output: 1 2 5 6 9
- ▶ Neighbor: 6 (visited)
- ▶ Stack: [4, 3]

STEP 6

- ▶ Pop → **3**
- ▶ Visited: {1,2,5,6,9,3}
- ▶ Output: 1 2 5 6 9 3
- ▶ Neighbors: 1,7 → push 7
- ▶ Stack: [4, 7]

STEP 7

- ▶ Pop → **7**
- ▶ Visited: {1,2,5,6,9,3,7}
- ▶ Output: 1 2 5 6 9 3 7
- ▶ Neighbors: 3,10 → push 10
- ▶ Stack: [4, 10]

STEP 8

- ▶ Pop → **10**
- ▶ Visited: {1,2,5,6,9,3,7,10}
- ▶ Output: 1 2 5 6 9 3 7 10
- ▶ Neighbors: 7,8 → push 8
- ▶ Stack: [4, 8]

STEP 9

- ▶ Pop → **8**
- ▶ Visited: {1,2,5,6,9,3,7,10,8}
- ▶ Output: 1 2 5 6 9 3 7 10 8
- ▶ Neighbors: 4,10,6 → 4 is unvisited → push 4
- ▶ Stack: [4, 4]

STEP 10

- ▶ Pop → **4** (first)
- ▶ Visited: {1,2,5,6,9,3,7,10,8,4}
- ▶ Output: 1 2 5 6 9 3 7 10 8 4
- ▶ Neighbors: 1,8 → already visited
No more pushes.
- ▶ Stack: [4]

STEP 11

- ▶ Pop → **4** (duplicate from earlier)
→ Already visited, ignore.
- ▶ Stack: []

▶ FINAL DFS TRAVERSAL

- ▶ 1 → 2 → 5 → 6 → 9 → 3 → 7 → 10 → 8 → 4

DFS

- We use nodes: **A, B, C, D, E, F, G, H, I, J**
- **Adjacency List (Directed Edges)**
 - $A \rightarrow B, C, D$
 - $B \rightarrow E, F$
 - $C \rightarrow F, G$
 - $D \rightarrow C, H$
 - $E \rightarrow I$
 - $F \rightarrow I, J$
 - $G \rightarrow J$
 - $H \rightarrow F$
 - $I \rightarrow$ (no outgoing edges)
 - $J \rightarrow H$

Algorithm Reminder

Use **Stack**

Process node when **popped**

Push neighbors in **reverse order** to maintain correct order

DFS

▶ Start

- ▶ Stack: [A]
- ▶ Visited: {}

Pop A

- ▶ Visited = {A}
- ▶ Push: D, C, B (reverse of $A \rightarrow B, C, D$)
- ▶ Stack = [D, C, B]

Pop B

- ▶ Visited = {A, B}
- ▶ Push: F, E
- ▶ Stack = [D, C, F, E]

Pop E

- ▶ Visited = {A, B, E}
- ▶ Push: I
- ▶ Stack = [D, C, F, I]

Pop I

- ▶ Visited = {A, B, E, I}
- ▶ (no outgoing edges)
- ▶ Stack = [D, C, F]

Pop F

- ▶ Visited = {A, B, E, I, F}
- ▶ Push: J, I (I already visited)
- ▶ Stack = [D, C, J]

Pop J

- ▶ Visited = {A, B, E, I, F, J}
- ▶ Push: H
- ▶ Stack = [D, C, H]

Pop H

- ▶ Visited = {A, B, E, I, F, J, H}
- ▶ Push: F (already visited)
- ▶ Stack = [D, C]

DFS

Pop C

- ▶ Visited = {A, B, E, I, F, J, H, C}
- ▶ Push: G, F
- ▶ Stack = [D, G]

Pop G

- ▶ Visited = {A, B, E, I, F, J, H, C, G}
- ▶ Push: J (already in visited)
- ▶ Stack = [D]

Pop D

- ▶ Visited = {A, B, E, I, F, J, H, C, G, D}
- ▶ Push: H, C (both already visited)
- ▶ Stack = []

FINAL DFS ORDER

A → B → E → I → F → J → H → C → G → D



Feature	BFS	DFS
Data Structure	Queue	Stack / Recursion
Traversal	Level-wise	Depth-wise
Shortest Path	✓ Yes	✗ No
Memory	High (stores whole level)	Low
Applications	shortest path, bipartite	topological sort, cycle detection

Single-source shortest-path problem

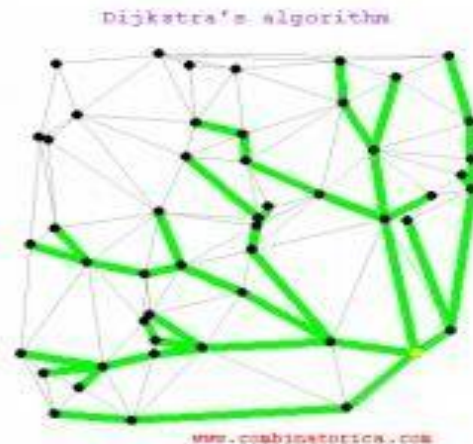
- There are multiple paths from a source vertex to a destination vertex
- Shortest path: the path whose total weight (i.e., sum of edge weights) is minimum
- Examples:
 - Austin->Houston->Atlanta->Washington: 1560 miles
 - Austin->Dallas->Denver->Atlanta->Washington: 2980 miles

Single-source shortest-path problem (cont.)

The problem of finding shortest paths from a source vertex v to all other vertices in the graph.

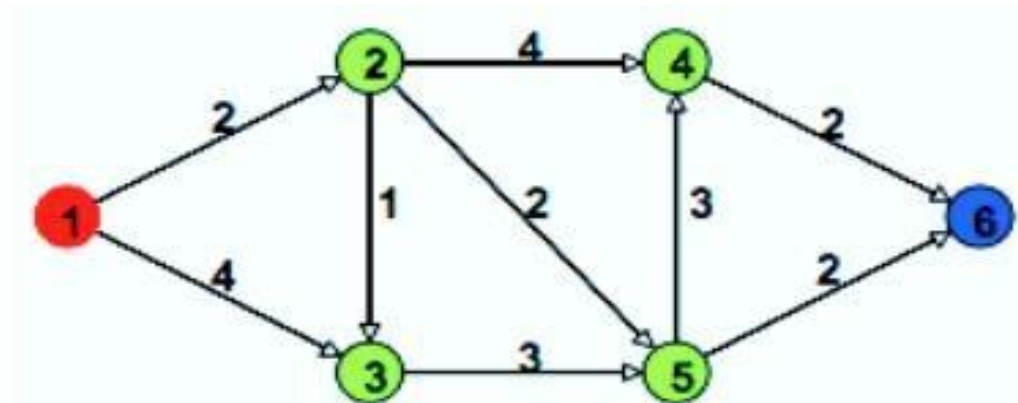
Weighted graph $G = (E, V)$

Source vertex $s \in V$ to all vertices $v \in V$



Dijkstra's Algorithm

- Solution to the single-source shortest path problem in graph theory
 - ▣ Both directed and undirected graphs
 - ▣ All edges must have nonnegative weights
 - ▣ Graph must be connected



Dijkstra's Algorithm

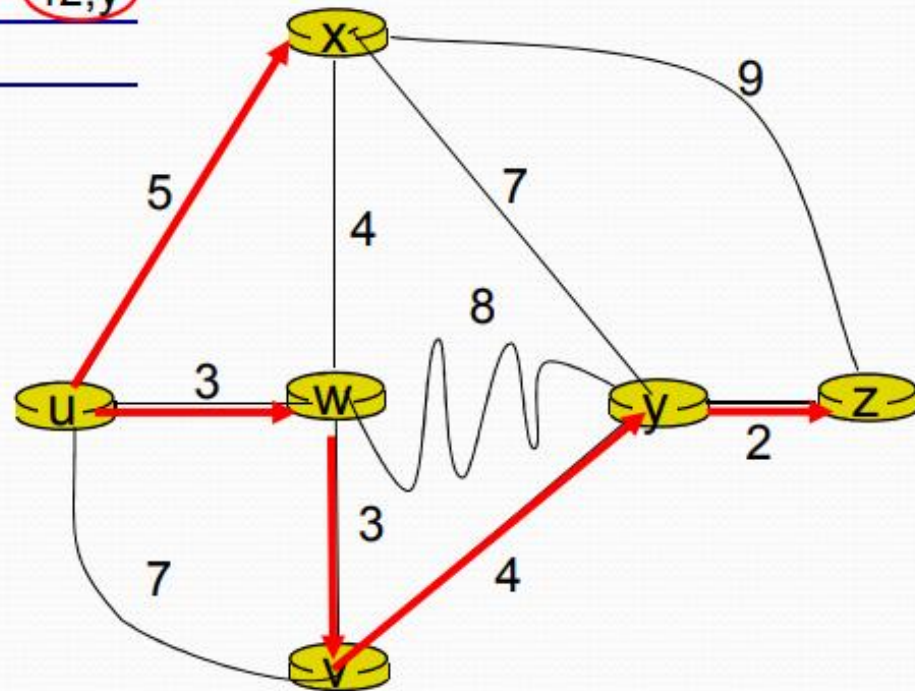
- Dijkstra's algorithm is used in problems relating to finding the shortest path.
- Each node is given a temporary label denoting the length of the shortest path *from* the start node *so far*.
- This label is replaced if another shorter route is found.
- Once it is certain that no other shorter paths can be found, the temporary label becomes a permanent label.
- Eventually all the nodes have permanent labels.
- At this point the shortest path is found by retracing the path backwards.

Dijkstra's algorithm: example

		D(v)	D(w)	D(x)	D(y)	D(z)
Step	N'					
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwx	6,w			11,w	14,x
3	uwxv				10,v	14,x
4	uwxvy					12,y
5	uwxvyz					

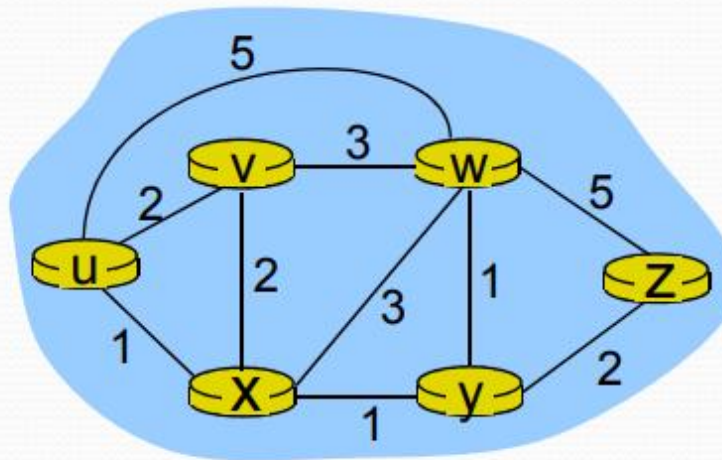
notes:

- ❖ construct shortest path tree by tracing predecessor nodes
- ❖ ties can exist (can be broken arbitrarily)



~~Dijkstra's algorithm: another example~~

Step	N'	D(v)	D(w)	D(x)	D(y)	D(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					



Pseudocode

```
dist[s] ← 0                                (distance to source vertex is zero)
for all v ∈ V - {s}
    do dist[v] ← ∞                          (set all other distances to infinity)
S ← ∅                                        (S, the set of visited vertices is initially empty)
Q ← V                                       (Q, the queue initially contains all vertices)
while Q ≠ ∅                                (while the queue is not empty)
do u ← mindistance(Q, dist)                (select the element of Q with the min. distance)
    S ← S ∪ {u}                            (add u to list of visited vertices)
    for all v ∈ neighbors[u]
        do if dist[v] > dist[u] + w(u, v)   (if new shortest path found)
            then d[v] ← d[u] + w(u, v)     (set new value of shortest path)
                                            (if desired, add traceback code)
return dist
```


Topological Sort

- ▶ **Topological Sort** is an ordering of vertices in a **directed acyclic graph (DAG)** such that:
 - For every directed edge $u \rightarrow v$ vertex u comes before v in the ordering.

► **Key Points:**

1. Topological sort is applicable only to **DAGs** (Directed Acyclic Graphs).
2. It represents a linear ordering of tasks based on dependencies.
3. A graph must not contain cycles for a topological sort to exist.

Applications



- **Task Scheduling:** When tasks depend on other tasks to be completed first.
- **Build Systems:** Order of compiling files in a program where some files depend on others.
- **Dependency Resolution:** In package managers (e.g., pip, npm), resolving dependencies of packages.

Properties of DAG

- A **DAG** is a directed graph with no cycles.
- If a graph contains a cycle, **topological sort is not possible**

Topological Sort (Two Algorithms)

- ▶ Kahn's Algorithm
- ▶ DFS based algorithm
- ▶ We will study DFS based Algo

Topological Sorting

- Create a graph with **n** vertices and **m**-directed edges.
- Initialize a stack and a visited array of size **n**.
- For each unvisited vertex in the graph, do the following:
 - Call the DFS function with the vertex as the parameter.
 - In the DFS function, mark the vertex as visited and recursively call the DFS function for all unvisited neighbors of the vertex.
 - Once all the neighbors have been visited, push the vertex onto the stack.
- After all, vertices have been visited, pop elements from the stack and append them to the output list until the stack is empty.
- The resulting list is the topologically sorted order of the graph.

Topological Sorting

Maintain:

- ✓ A **visited[]** array
- ✓ A **stack** (stores ordering)

For every unvisited vertex:

→ Call **DFS(vertex)**

Inside DFS(u):

Mark u as visited

For each neighbor v of u:

→ If not visited, DFS(v)

After exploring all neighbors:

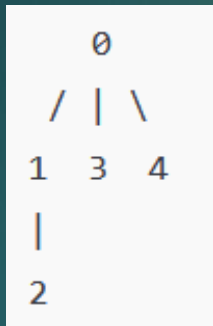
→ **Push u into the stack**

After all DFS calls:

→ Pop all elements from stack

→ This is the **topological ordering**

Dry Run Example



Construct Adjacency List First

Vertex	Adjacency List
0	{1, 3, 4}
1	{2}
2	{}
3	{}
4	{}

Dry Run Example

1. Start DFS at Vertex 0

1. Visit 0, mark it visited.
2. Explore 0's neighbors in order:
 1. 1:
 1. Visit 1, mark it visited.
 2. Explore 1's neighbor 2:
 1. Visit 2, mark it visited. Push 2 to the stack.
 3. Push 1 to the stack.
 2. 3:
 1. Visit 3, mark it visited. Push 3 to the stack.
 3. 4:
 1. Visit 4, mark it visited. Push 4 to the stack.
3. Push 0 to the stack.

2. Reverse the Stack (Stack works in LIFO)

1. Pop items

Sequence: 0,4,3,1,2

Topological Sort

- ▶ Not a unique solution
- ▶ Some of the Other possibilities

0,1,2,3,4

0,1,3,4,2

0,3,1,2,4

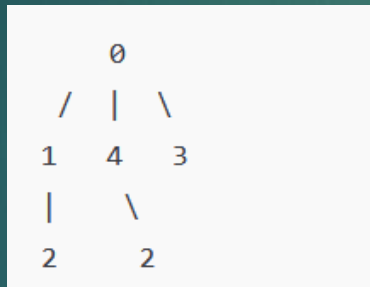
0,3,4,1,2

0,4,1,2,3

Remember

- ▶ During the DFS traversal, we push nodes onto the stack only after all their dependencies (outgoing neighbors) have been visited.
- ▶ When the DFS completes, the stack naturally contains the nodes in reverse order of completion, which is the required topological order.

Another Example



Adjacency List

Vertex	Adjacency List
0	{1, 4, 3}
1	{2}
2	{}
3	{}
4	{2}

Dry Run

- ▶ Initialization
 - ▶ Visited array: To keep track of visited nodes. Initially, all nodes are unvisited:
 - ▶ Visited=[false,false,false,false,false]
 - ▶ Stack: To store the topological order.
- ▶ Start DFS from Vertex 0
 - ▶ Mark 0 as visited:
 - ▶ Visited=[true,false,false,false,false]
 - ▶ Explore neighbors of 0 in order: 1,4,3.

- ▶ DFS on Vertex 1

- ▶ Mark 1 as visited:
- ▶ Visited=[true,true,false,false,false]
- ▶ Explore neighbor of 1:2.

- ▶ DFS on Vertex 2

- ▶ Mark 2 as visited:
- ▶ Visited=[true,true,true,false,false]
- ▶ 2 has no neighbors. Push 2 to the stack:
- ▶ Stack=[2]

- 
- ▶ Backtrack to Vertex 1
 - ▶ All neighbors of 1 are visited. Push 1 to the stack:
 - ▶ Stack=[1,2]
 - ▶ DFS on Vertex 4
 - ▶ Back at 0, proceed to 4. Mark 4 as visited:
 - ▶ Visited=[true,true,true,false,true]
 - ▶ Explore neighbor of 4: 2.
 - ▶ 2 is already visited, so do nothing.
 - ▶ Push 4 to the stack:
 - ▶ Stack=[4,1,2]

- 
- ▶ DFS on Vertex 3
 - ▶ Back at 0, proceed to 3. Mark 3 as visited:
 - ▶ Visited=[true,true,true,true,true]
 - ▶ 3 has no neighbors. Push 3 to the stack:
 - ▶ Stack=[3,4,1,2]
 - ▶ Backtrack to Vertex 0
 - ▶ All neighbors of 0 are visited. Push 0 to the stack:
 - ▶ Stack=[0,3,4,1,2]
 - ▶ Final Topological Order
 - ▶ The stack now contains the topological order of the graph:
 - ▶ 0,3,4,1,2
 - ▶ Graph Traversal Explanation
 - ▶ 0 must come before 1,4,3. 1 must come before 2. 4 must also come before 2.



Code Of Algorithms



```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
```

```
void BFS(int start, vector<vector<int>>& adj, int n) {
```

```
    vector<bool> visited(n+1, false); // keeps track of visited nodes
    queue<int> q;                       // queue for BFS
```

```
    visited[start] = true;              // mark starting node visited
    q.push(start);                      // push starting node to queue
```

```
    cout << "BFS Traversal: ";
```

```
    while (!q.empty()) {                // run until queue becomes empty
        int u = q.front();              // get front element
        q.pop();                        // remove it
        cout << u << " ";              // print visited node
```

```
        // explore all neighbors of u
        for (int v : adj[u]) {
            if (!visited[v]) {          // if neighbor not visited
                visited[v] = true;      // mark visited
                q.push(v);              // push into queue
            } } } }
```



```
#include <iostream>
#include <vector>
#include <stack>
using namespace std;
```

```
void DFS(int start, vector<vector<int>>& adj, int n) {

    vector<bool> visited(n+1, false); // visited array
    stack<int> st;                     // stack for DFS

    st.push(start);                   // start DFS with the start node

    cout << "DFS Traversal: ";

    while (!st.empty()) {             // continue until stack is empty
        int u = st.top();              // get top element
        st.pop();                     // remove it from stack

        if (!visited[u]) {            // process only if unvisited
            visited[u] = true;
            cout << u << " ";

            // Push neighbors in reverse order so leftmost is processed first
            for (int i = adj[u].size() - 1; i >= 0; i--) {
                int v = adj[u][i];
                if (!visited[v])      // push only unvisited neighbors
                    st.push(v);
            }
        }
    }
}
```



```
#include <iostream>
#include <vector>
#include <stack>
using namespace std;
```

```
// DFS function for topological sorting
```

```
void topoDFS(int u, vector<vector<int>>& adj, vector<bool>& visited, stack<int>&
st) {
```

```
    visited[u] = true;           // mark node as visited
```

```
    for (int v : adj[u]) {       // explore all outgoing edges from u
        if (!visited[v])
            topoDFS(v, adj, visited, st); // recursively call DFS
    }
```

```
    st.push(u);                  // push node after exploring children
}
```

```
void topologicalSort(vector<vector<int>>& adj, int n) {
```

```
    vector<bool> visited(n+1, false); // visited tracker
    stack<int> st;                     // stack will store topo order
```

```
    // run DFS from all unvisited nodes
    for (int i = 1; i <= n; i++) {
        if (!visited[i])
            topoDFS(i, adj, visited, st);
    }
```



```
// final topological order (stack -> left to right)
cout << "Topological Order: ";
while (!st.empty()) {
    cout << st.top() << " ";
    st.pop();
}
}

int main() {
    int n = 10;
    vector<vector<int>> adj(n+1);

    // DAG connections
    adj[1] = {2,3};
    adj[2] = {5,6};
    adj[3] = {7};
    adj[4] = {8};
    adj[6] = {9};
    adj[7] = {8,10};

    topologicalSort(adj, n);

    return 0;
}
```



```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
```

```
void dijkstra(int source, vector<vector<pair<int,int>>> &adj, int n) {
    vector<int> dist(n+1, 1e9); // distance array
    vector<bool> visited(n+1, false);

    // Min-heap: (distance, node)
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;

    dist[source] = 0;
    pq.push({0, source}); // push starting node

    while (!pq.empty()) {
        auto [d, u] = pq.top(); // node with smallest distance
        pq.pop();

        if (visited[u]) continue; // skip already processed nodes
        visited[u] = true;

        for (auto &[v, w] : adj[u]) { // explore neighbors
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w; // update if shorter path found
                pq.push({dist[v], v});
            }
        }
    }
}
```

```
cout << "Shortest distances from source " << source << ":\n";
for (int i = 1; i <= n; i++)
    cout << "Node " << i << " = " << dist[i] << "\n";
}
```

```
int main() {
    int n = 6; // number of nodes
    vector<vector<pair<int,int>>> adj(n+1);

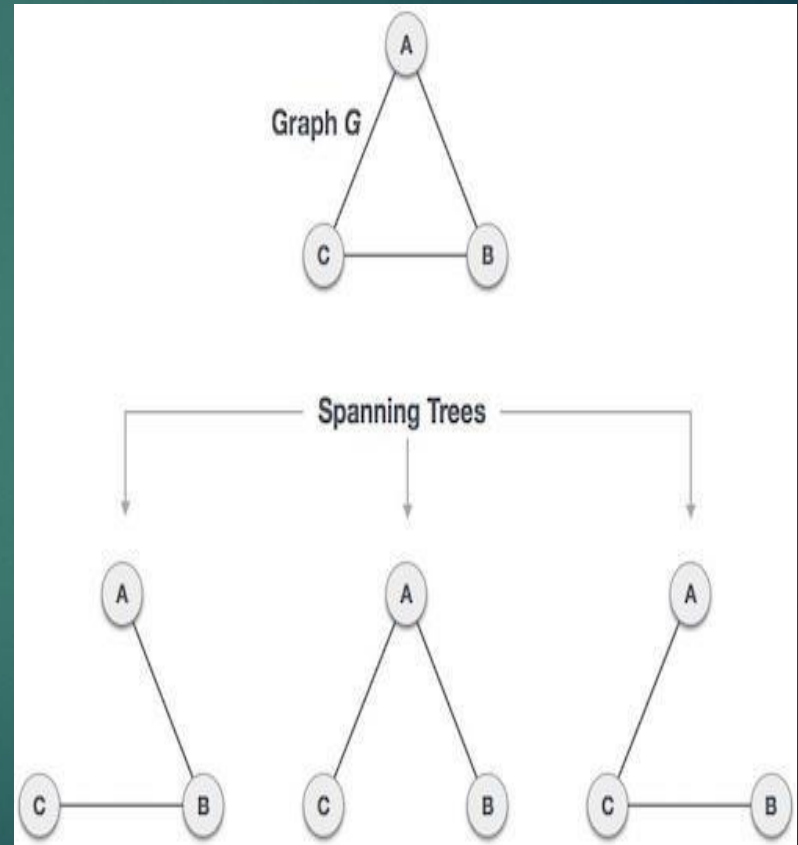
    // Weighted graph edges: u -> v (weight)
    adj[1] = {{2,4}, {3,2}};
    adj[2] = {{3,5}, {4,10}};
    adj[3] = {{5,3}};
    adj[5] = {{4,4}};
    adj[4] = {{6,11}};
    adj[5] = {{6,2}};

    dijkstra(1, adj, n);

    return 0;
}
```

Spanning Tree

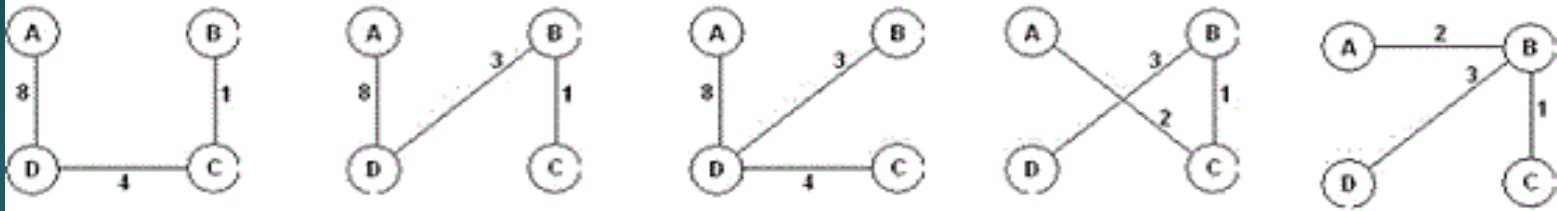
- Given an undirected and connected graph $G(V,E)$
- A spanning tree is a sub graph of G , which must include all the vertices.



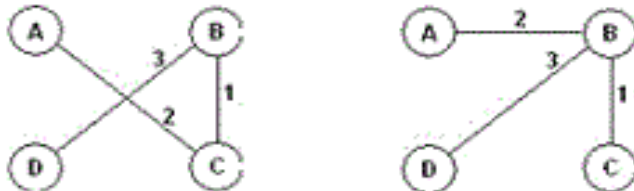
Minimum Spanning Tree

- A minimum spanning tree of a connected undirected weighted graph.
- It has a subset of the edges from the original graph that connects all the vertices together, without any cycles and with the minimum possible total edge weight.

Has 16 spanning trees. Some are:



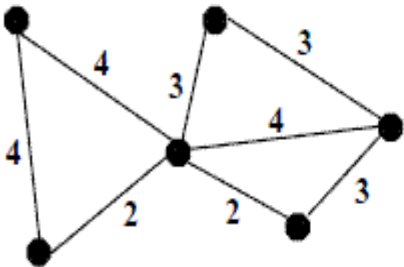
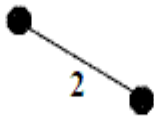
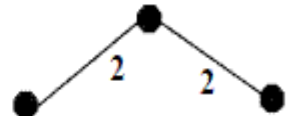
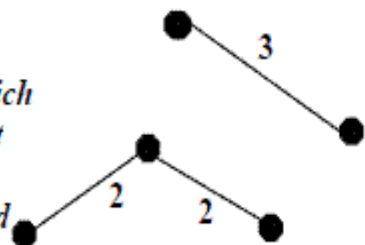
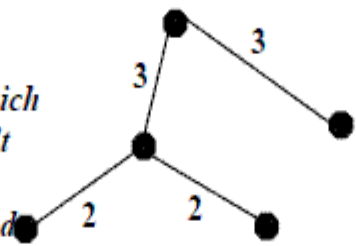
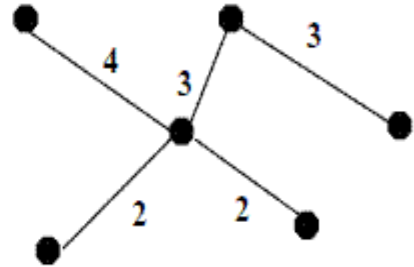
The graph has two minimum-cost spanning trees, each with a cost of 6:



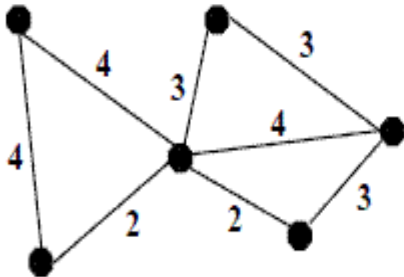
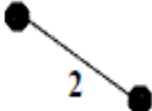
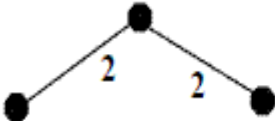
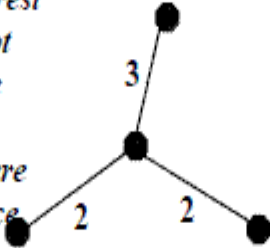
Algorithms for MST

- Kruskal's Algorithm
- Prim's Algorithm

Kruskal's Algorithm

1 <i>Given a network.....</i> 	2 <i>Choose the shortest edge (if there is more than one, choose any of the shortest).....</i> 	3 <i>Choose the next shortest edge and add it.....</i> 
4 <i>Choose the next shortest edge which wouldn't create a cycle and add it.</i> 	5 <i>Choose the next shortest edge which wouldn't create a cycle and add it.</i> 	6 <i>Repeat until you have a minimal spanning tree.</i> 

Prim's Algorithm

1 <i>Given a network.....</i> 	2 <i>Choose a vertex</i> 	3 <i>Choose the shortest edge from this vertex.</i> 
4 <i>Choose the nearest vertex not yet in the solution.</i> 	5 <i>Choose the next nearest vertex not yet in the solution, when there is a choice choose either.</i> 	6 <i>Repeat until you have a minimal spanning tree.</i> 